
CATANBENCH: A REAL-TIME BENCHMARK FOR STRATEGIC REASONING IN SETTLERS OF CATAN

Vayun Gupta, Sharayu Janga, Dhruv Malpani, Rudy Juarez-Ovallos
University of Illinois Urbana Champaign

ABSTRACT

We present CatanBench, a benchmark for evaluating strategic decision making in the board game Settlers of Catan. Unlike benchmarks that primarily score agents by final game outcome, CatanBench evaluates individual decisions at the moment they are made. The benchmark covers short-term tactical moves, medium-term resource and build prioritization, and long-term strategic commitments such as pursuing Longest Road or Largest Army. Each decision is compared against the best legal alternative available in the current game state using heuristics. CatanBench is implemented inside a complete Node.js/React Catan environment with a Socket.IO multiplayer backend and a Python tool API for agents. During live gameplay, agent actions automatically trigger task evaluators, persist task scores, and support both real-time telemetry and offline aggregate analysis.

1 INTRODUCTION

Many game benchmarks evaluate whether an agent wins, but this is often too sparse to diagnose strategic reasoning. In Settlers of Catan, a final win or loss is shaped by hidden information, stochastic dice rolls, turn order, opponent negotiation, and long delayed consequences. An agent can make a strong settlement placement and still lose to unlucky dice; it can also win after repeatedly making fragile trades because opponents failed to punish them. This makes outcome-only evaluation a poor fit for studying whether an agent understands the quality of the choices available at each decision point.

CatanBench addresses this problem by evaluating decisions during live gameplay. The benchmark covers reasoning across short-, medium-, and long-term strategic moves. Short-term tasks include tactical decisions such as where to place a road, which opponent to rob, and whether to accept a trade. Medium-term tasks evaluate turn-level prioritization, including whether to build now or save, whether to buy a development card, and how to discard after a seven. Long-term tasks evaluate strategic commitments such as opening settlement placement, pursuing Longest Road, and pursuing Largest Army. Each task scores the chosen action against the best legal alternative currently available, rather than against a fixed answer that ignores the board state.

Existing benchmarks are insufficient for this setting. GameBench and Kaggle Game Arena treat match outcomes as proxies for strategy quality. CatanBench scores the quality of the decision directly in each legal action. Most benchmarks isolate the search for perfect information or single step planning and miss Catan’s mixed information structure simultaneous management of board, private hands, stochastic rolls and open ended negotiation. Language agent benchmarks typically evaluate final textual answers rather than tool-mediated actions in a changing multiplayer environment.

Our contribution is a 20-task benchmark for Catan decision making that runs during real games. The benchmark evaluates each decision against the legal alternatives available at that exact moment of gameplay using deterministic heuristic rubrics. It records both task-level quality and game-level performance, enabling analysis of whether agents win, how they win, and which decision families cause failure. CatanBench runs on a complete Node.js/React implementation of Settlers of Catan with a Socket.IO multiplayer backend, adapted from the open-source Catan Online project (Doshi, 2026). Agents interact through a Python tool API. Each agent action automatically triggers the relevant evaluator, stores the score, and contributes to real-time telemetry and offline aggregate reports.

2 RELATED WORK

CatanBench is related to recent game-based benchmarks for evaluating strategic reasoning in language-model agents. GameBench evaluates the strategic reasoning abilities of LLM agents in game environments and is closest to CatanBench in its emphasis on strategic rather than purely static reasoning (Costarelli et al., 2024). CivBench evaluates AI agents in FreeCiv, a long-horizon adversarial environment designed to stress planning quality, adaptation under uncertainty and pressure, persistent strategy identity, execution efficiency, and stability across complete match trajectories rather than one-turn snapshots (Halevy, 2026). Kaggle Game Arena provides a broader framework around game environments, model harnesses, visualizers, and leaderboards, ranking models by outcome-oriented metrics such as Elo while using games as structured tests of strategic reasoning, long-term planning, and dynamic adaptation (Risdal, 2025).

CatanBench differs from these benchmarks by scoring decision quality during a full game instead of relying primarily on match outcome, Elo, or trajectory-level summaries. The benchmark is not only a Catan environment; it is an instrumentation layer over gameplay that records whether an agent’s selected legal action is close to the best legal alternative under a task-specific rubric. This fills a complementary gap: CatanBench evaluates situated strategic judgment under changing board state, stochastic dice outcomes, hidden opponent resources, and negotiation, while preserving dense per-decision feedback across short, medium, and long-term Catan tasks.

3 BENCHMARK DESIGN

3.1 TASK DOMAIN AND SCOPE

CatanBench covers four categories of Catan decisions:

1. Spatial placement tasks evaluate settlement and road locations.
2. Resource management tasks evaluate trading, discarding, bank trades, and build-versus-save decisions.
3. Opponent interaction tasks evaluate robber placement, victim selection, trade targeting, counteroffers, and leader blocking.
4. Strategic commitment tasks evaluate whether an agent should pursue Longest Road or Largest Army over several moves.

The benchmark is designed to test decision quality rather than only game outcome. This includes individual legal decisions made during real gameplay, their task-specific scores, pass/fail thresholds, and aggregate performance over games. Natural language explanation grading is out of scope for this benchmark. No existing benchmark framework was used to design CatanBench.

3.2 TASK SUITE

Table 1 provides a brief overview of the 20 tasks by difficulty and reasoning horizon. We summarize 3 representative tasks below, while a full comprehensive description of all 20 tasks can be found in the Appendix.

Initial settlement location selection. The input is a setup-phase game-state snapshot containing the board layout, legal settlement vertices, adjacent hex resources and dice numbers, port locations, occupied vertices, and legal outgoing road options. The output is a legal settlement vertex. Each candidate is scored as:

$$\text{score} = 0.40 \cdot \text{production} + 0.25 \cdot \text{diversity} + 0.20 \cdot \text{expansionAccess} + 0.15 \cdot \text{portSynergy}. \quad (1)$$

Production uses Catan dice probability weights; diversity measures distinct adjacent resource types; expansion access counts future legal outgoing road choices; and port synergy rewards 2:1 or 3:1 port access. The selected vertex is compared against the best legal vertex. The current pass threshold is 0.75 because opening placements are highly consequential but multiple strong starts can be defensible. This task is challenging because the first settlement shapes resource income, trade dependence, expansion paths, and the feasibility of future ports before the agent has observed any dice outcomes.

Discard strategy after seven. The input is a player hand after a seven is rolled, the required discard count, the current board state, legal future build plans, and resource need/surplus estimates. The output is a legal discard bundle. The evaluator enumerates legal discard bundles and scores the retained hand:

$$\text{retainedHandValue} = 0.55 \cdot \text{bestAffordable} + 0.30 \cdot \text{bestFuture} + 0.15 \cdot \text{diversity}. \quad (2)$$

The final score is $\text{RetainedHandValue} / \text{BestRetainedValue}$, and the pass threshold is 0.95. This task is challenging because the agent must reason about opportunity cost: the discarded cards matter less than whether the remaining hand preserves an immediate build, a near-term plan, and enough diversity to avoid bottlenecks.

3.3 TASK DESIGN PRINCIPLES

The central design principle is comparison against the best legal alternative. This avoids treating a reasonable but second-best move as fully wrong and makes scores robust to the current board state. We use component-weighted rubrics when the decision has interpretable factors. For example, settlement placement assigns 0.40 weight to production because early income determines the opening trajectory, and 0.20 to expansion access because path-dependent road investments are difficult to recover from if the settlement is geometrically constrained.

Coverage was achieved by mapping the actions a player can take in a turn to spatial placement, card play, negotiation and trading, build prioritization, discarding, and strategic commitment. Difficulty was calibrated by horizon. Easy tasks are immediate tactical decisions with direct legal alternatives. Medium tasks require turn-level prioritization across resources and possible plans. Hard tasks require opening or multi-turn strategic reasoning, where immediate scoring can be misleading and delayed evaluation is needed.

3.4 DIFFICULTY DISTRIBUTION

Difficulty was determined based on the decision horizon and the amount of strategic context required. Easy tasks are short-term decisions with immediate consequences and relatively direct legal alternatives, such as road placement, robber victim selection, or accepting a trade. Medium tasks require turn-level prioritization across competing plans, such as deciding whether to build now or save, whether to buy a development card, or which resources to discard after a seven. Hard tasks involve opening or multi-turn strategic commitments, such as initial settlement placement, pursuing Longest Road, or pursuing Largest Army, because their value depends on future board development, opponent behavior, resource availability, and delayed payoff rather than a single immediate action. Table 1 summarises the task difficulty levels, and distribution across easy, medium and hard tasks.

Difficulty		Early game	Mid game	End game
Easy Immediate tactical decisions		road-placement-direction robber-placement	discard-strategy-after-seven	year-of-plenty-card-playing-decision
Medium Turn-level prioritization		bank-trade-decision robber-victim-selection	accept-or-reject-trade-offers select-targeted-trade-partner generate-trade-offers settlement-location-selection monopoly-card-playing-decision city-vs-settlement-vs-road-prioritization development-card-purchase-decision	knight-card-playing-decision road-building-card-playing-decision
Hard Involves long-term strategy		initial-settlement-location-selection	build-vs-save-decision generate-counter-trade-offer	decide-pursue-longest-road decide-pursue-largest-army

Table 1: Difficulty distribution for the 20 CatanBench tasks, grouped by game horizon. Early tasks are most common in setup and opening play, mid-game tasks focus on turn-level resource and build prioritization, and end-game tasks emphasize longer-horizon strategic commitments.

4 EVALUATION PROTOCOL

Agents are evaluated in benchmark-enabled Catan games. Each action is recorded with metadata including benchmark ID, agent ID, agent version, map layout, opponent policy set, scenario tags, selected option, task score, task threshold, latency, and legality. Task scores are computed and task succeeds when its score exceeds the task-specific threshold. The benchmark has the following metrics: win rate, overall score, game play score, reasoning score, operations score, task success rate, average final victory points, average rounds to win, average task score, and average latency per turn. We omit robustness, consistency, and generalization from the main score because they require reliable scenario-slice coverage across maps, seats, and opponent policies. Primary metrics aggregate across gameplay, reasoning, and operational performance.

Before aggregation, each metric is normalized to the range $[0, 1]$ so that higher is always better. Win rate and task success rate are already normalized. Average final victory points is divided by 10. Average rounds to win is normalized as $\text{clamp}((20 - \text{rounds})/12)$, so faster wins receive higher scores. Average latency per turn receives score 1 below 2 seconds, score 0 above 15 seconds, and linearly decays between those values.

The overall score is the weighted sum of the normalized metrics:

$$\begin{aligned} \text{overallScore} = & 0.30 \text{winRate} + 0.15 \text{averageFinalVictoryPoints} \\ & + 0.15 \text{averageRoundsToWin} + 0.30 \text{taskSuccessRate} \\ & + 0.10 \text{averageLatencyPerTurn}. \end{aligned}$$

The category scores are reported to make the overall score easier to interpret. The gameplay score is the average of normalized win rate, normalized final victory points, and normalized rounds to win:

$$\text{gameplayScore} = \frac{\text{winRate} + \text{averageFinalVictoryPoints} + \text{averageRoundsToWin}}{3}.$$

The `reasoningScore` is the normalized task success rate. The `operationsScore` is the normalized latency score. Reports should include overall score, task success rate, average task score, per-task breakdowns, win rate, average final victory points, average rounds to win, and latency per turn.

5 BENCHMARK VALIDATION

We conducted testing by having humans play complete games while CatanBench logged tasks in real time. These games were used to check whether task logs occurred during the intended move and whether the heuristic rankings aligned with intuitive Catan play. For example, the settlement rubric ranked high-production, diverse, expandable locations highly, and the robber placement heuristic generally matched common denial strategies that target the leader while avoiding self-harm.

Task quality is supported by deterministic task specifications. Each of the 20 tasks takes a deterministic input and a scoring function. Production weights, resource diversity, expansion access, port synergy, build-plan values, and resource need/surplus are reused consistently across tasks. For long-term tasks, we moved from immediate scoring to delayed four-to-six-turn simulation.

6 CONCLUSION

CatanBench is a real-time benchmark for strategic reasoning in Settlers of Catan. It evaluates 20 live gameplay tasks across tactical decisions, resource prioritization, negotiation, and long-horizon commitments. By scoring decisions against the best legal alternative at the moment of play, CatanBench provides denser and more diagnostic feedback than final win rate alone. We expect the benchmark to help researchers study agents that must reason under hidden information, stochastic outcomes, negotiation, and delayed strategic consequences. Future work includes larger human and agent baselines, stronger negotiation modeling, additional scenario suites, and calibration against expert Catan play.

REFERENCES

- Viral Doshi. Catan Online: A real-time multiplayer implementation of Settlers of Catan. GitHub repository, commit 3a0a6b815ff999adf5fd5802fa3df9b99833fc2d, January 1, 2026. github.com/Viral-Doshi/catan.
- Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, and Arjun Yadav. GameBench: Evaluating strategic reasoning abilities of LLM agents. *arXiv preprint arXiv:2406.06613v1*, June 7, 2024. arxiv.org/html/2406.06613v1.
- Matan Halevy. Introducing CivBench Season 001. ClashAI, February 25, 2026. clashai.live/blog/ai/introducing-civbench-season-001.
- Meg Risdal. Introducing Kaggle Game Arena. Kaggle, August 4, 2025. <https://www.kaggle.com/blog/introducing-game-arena>.

A BENCHMARK TASK EXPLANATIONS

A.1 DESIGN PRINCIPLES

The benchmark uses three families of tasks:

- **Short-term tasks:** immediate tactical choices with direct board consequences.
- **Medium-term tasks:** action prioritization and resource conversion decisions.
- **Long-term tasks:** strategic race commitments whose value unfolds over several turns.

The main design goal is to compare decisions with the best legal alternative available at that moment (based on a heuristic approach), while keeping a threshold value to determine success or failure on the individual task. This system rewards near-optimal play when several choices are reasonable, and avoids placing fail labels on decisions that were reasonable but not perfectly aligned with the benchmark heuristic.

A.2 SHARED HEURISTIC BUILDING BLOCKS

A.2.1 PRODUCTION EXPECTANCY

Some dice numbers have a higher probability of being rolled than others. Specifically, the probability weights are:

$$2, 12 \mapsto 1, \quad 3, 11 \mapsto 2, \quad 4, 10 \mapsto 3, \quad 5, 9 \mapsto 4, \quad 6, 8 \mapsto 5.$$

7 is a number used for the robber and does not produce resources. To compute the expected value of the resources gained on the next turn from a current settlement vertex,

$$\text{productionWeight} = \sum \text{adjacent_hex_probability_weights},$$

$$\text{resourceProductionExpectancy} = \min \left(1, \frac{\text{productionWeight}}{15} \right).$$

The denominator 15 is the theoretical maximum because a vertex touches at most three hexes and the maximum per-hex weight is 5.

A.2.2 RESOURCE DIVERSITY

Each settlement vertex is adjacent to up to three hexes, which can produce up to three different resource types (`distinct_adjacent_resource_types`).

Resource diversity is defined as

$$\text{resourceDiversity} = \min \left(1, \frac{\text{distinct_adjacent_resource_types}}{3} \right).$$

It matters because diverse starts reduce dependency on trading, improve build robustness, and lower the risk of bottlenecking on a single missing resource.

A.2.3 EXPANSION ACCESS

In crowded boards, some settlement locations have more future legal road placements than others, which translates into better expansion potential and better survivability in case of unfavorable dice rolls or robber placements. For a settlement candidate, the evaluator computes incident edges, keeps only edges where a road could legally be placed, and defines

$$\text{expansionAccess} = \min \left(1, \frac{\text{legal_outgoing_edges}}{3} \right).$$

A.2.4 PORT SYNERGY

We use a discrete port heuristic:

- specific 2:1 port $\rightarrow 1.0$
- generic 3:1 port $\rightarrow 0.7$
- no port $\rightarrow 0.0$

Ports matter because they convert surplus resources into missing ones; specific ports are usually stronger due to higher exchange efficiency.

A.2.5 BUILD PLAN BASE SCORES

Several tasks construct plan candidates for `road`, `settlement`, `city`, and `devCard`. Each plan has a `baseScore`: the current heuristic value of that build family before applying affordability or distance-to-completion discounts. The shared definitions are:

- `road`: the maximum score among currently legal road placements under the road-placement rubric from Section A.3.3.
- `settlement`: the maximum score among currently legal settlement placements under the settlement rubric from Section A.3.2.
- `city`: the maximum score among legal city upgrades using $0.65 \cdot \text{production} + 0.10 \cdot \text{diversity} + 0.05 \cdot \text{portSynergy} + 0.20$.
- `devCard`: 0 if the development-card deck is unavailable; otherwise $\max(0.2, 0.55 \cdot (0.5 + 0.5 \cdot \text{cityPlanScore}))$, where `cityPlanScore` is the current best city-upgrade score computed with the city formula in this list.

A.2.6 RESOURCE NEED AND SURPLUS

Resource need and surplus are used in trade and bank-trade scoring.

For each candidate plan and resource r , `cost[r]` is the number of cards of resource r required by the plan, `hand[r]` is the player's current count of resource r , and

$$\text{missing}[r] = \max(0, \text{cost}[r] - \text{hand}[r]).$$

`totalMissing` is the total number of resource cards still needed for the plan:

$$\text{totalMissing} = \sum_r \text{missing}[r].$$

Each plan is weighted according to how many resources are needed to achieve the plan:

$$\text{planWeight} = \frac{\text{baseScore}}{1 + 0.75 \cdot \text{totalMissing}},$$

where `baseScore` uses the shared build-plan definition from Section A.2.5. For each resource r ,

$$\text{need}[r] = \text{planWeight} \cdot \frac{\text{missing}[r]}{\max(1, \text{totalMissing})}.$$

For surplus, the evaluator reserves part of the current hand for weighted plans:

$$\text{reserved}[r] = \min(\text{hand}[r], \sum \text{planWeight} \cdot \text{cost}[r]),$$

then treats any unreserved cards of resource r as surplus. The surplus map is normalized by the largest unreserved resource count in the hand:

$$\text{surplus}[r] = \frac{\max(0, \text{hand}[r] - \text{reserved}[r])}{\max(1, \max_s \max(0, \text{hand}[s] - \text{reserved}[s]))},$$

where s ranges over all resource types. This avoids earlier failure modes where a resource such as ore looked important merely because it was scarce in hand, even when the board strongly favored early road and settlement expansion.

A.3 TASK-BY-TASK EXPLANATIONS

A.3.1 INITIAL-SETTLEMENT-LOCATION-SELECTION

$\text{Score} = 0.40 \cdot \text{production} + 0.25 \cdot \text{diversity} + 0.20 \cdot \text{expansionAccess} + 0.15 \cdot \text{portSynergy}$.

This rubric uses the shared `production`, `diversity`, `expansionAccess`, and `portSynergy` definitions from Sections A.2.1–A.2.4. Production receives the highest weight because early income determines the opening trajectory, while the other terms preserve resource mix, expansion geometry, and port upside. The evaluator applies the rubric above to the shared settlement-quality quantities and clamps the final score to $[0, 1]$.

The threshold for `initial-settlement-location-selection` is 0.75 because opening settlement quality is highly consequential but multiple strong starts can be strategically defensible.

A.3.2 SETTLEMENT-LOCATION-SELECTION

This task uses the same scoring system as the initial-settlement task in Section A.3.1. The core difference between initial and subsequent location selection is that the initial settlement location selection task has a much higher weightage in the overall benchmark. This reflects the fact that the first settlement sets the foundation for the entire game and is a critical decision point, while subsequent settlement placements, although still important, are often more constrained by the existing board state and may have less impact on the overall game trajectory.

The threshold for `settlement-location-selection` is 0.90 because later settlement choices are evaluated against a smaller legal option set and should closely match the best available site.

A.3.3 ROAD-PLACEMENT-DIRECTION

$$\begin{aligned} \text{Score} = & 0.50 \cdot \text{futureReachability} + 0.35 \cdot \text{reachableValue} \\ & + 0.15 \cdot \text{blockingValue}. \end{aligned}$$

Where:

- `futureReachability` is the forward vertex’s onward legal-road count, clamped after division by 3.
- `reachableValue` is the forward vertex’s weighted settlement value:
 $\text{reachableValue} = 0.5 \cdot \text{production} + 0.3 \cdot \text{diversity} + 0.2 \cdot \text{portSynergy}$.
- `blockingValue` is the maximum blocking score over the candidate road’s two endpoints and all opponents present at those endpoints.

For each endpoint-opponent pair, the evaluator ignores endpoints occupied by the acting player and endpoints occupied by a different opponent. It then computes:

$O_{\text{branch}} = \text{open incident edges excluding the candidate road},$

$$\text{branchScarcity} = \text{clamp} \left(\frac{2 - O_{\text{branch}}}{2} \right),$$

$$\text{vertexOpportunity} = \begin{cases} 0.8, & \text{if the opponent already owns the endpoint,} \\ 1.0, & \text{if the endpoint is an open legal settlement site,} \\ 0.35, & \text{otherwise} \end{cases}$$

Let R_{adj} be the number of opponent roads adjacent to the endpoint, B_{opp} be 1 if the opponent owns the endpoint and 0 otherwise, L_{opp} be the opponent’s road length, and H_{opp} be 1 if the opponent has Longest Road and 0 otherwise.

$$\text{presenceStrength} = \text{clamp} \left(\frac{R_{\text{adj}} + B_{\text{opp}}}{2} \right),$$

$$\text{roadPressure} = \text{clamp} \left(\frac{L_{\text{opp}} + 2 \cdot H_{\text{opp}}}{8} \right).$$

The candidate blocking score is

$$\text{candidateBlocking} = \text{clamp}(0.50 \cdot \text{branchScarcity} + 0.30 \cdot \text{vertexOpportunity} + 0.10 \cdot \text{presenceStrength} + 0.10 \cdot \text{roadPressure}).$$

and `blockingValue` is the largest `candidateBlocking`, or 0 if no opponent is present at either endpoint. The current implementation therefore recognizes common denial plays without running a full tactical search. Future reachability is weighted highest because roads are primarily investment moves. Reachable intersection value matters because roads derive value from the future settlement points they unlock. Blocking value is tactical and useful, but not every road is a denial move.

The threshold for `road-placement-direction` is 0.80 because roads are path-dependent investment moves and several near-best routes can preserve similar future options.

A.3.4 KNIGHT-CARD-PLAYING-DECISION

Define:

- `robberThreatLevel` measures how costly the current robber placement is for the player.
- `resourceNeed` is the largest entry in the goal-aware resource-need map from Section A.2.6.
- `armyProgressRatio` measures progress toward Largest Army after playing the Knight:

$$\text{robberThreatLevel} = \begin{cases} \text{clamp}(\text{robberNumberWeight}/5), & \text{if the robber blocks the player} \\ 0.15, & \text{otherwise} \end{cases}$$

$$\text{resourceNeed} = \max_r \text{need}[r].$$

This captures how valuable a Knight steal is when the player is missing an important resource for a near-term plan. Let K_{self} be the player's current Knight count and K_{opp} be the largest opponent Knight count. Then:

$$\text{armyProgressRatio} = \text{clamp}\left(\frac{K_{\text{self}} + 1}{\max(3, K_{\text{opp}} + 1)}\right).$$

Playing a Knight is most valuable when the robber is harming the player, when stealing a needed resource is likely to matter, and when Largest Army progress matters. The score does not include resource hand size because the Knight card has already been purchased and playing it does not reduce seven-risk for the player's resource hand.

The live scores are

$$\begin{aligned} \text{playNowScore} &= 0.45 \cdot \text{robberThreatLevel} + 0.35 \cdot \text{resourceNeed} \\ &\quad + 0.20 \cdot \text{armyProgressRatio} \\ \text{holdScore} &= \text{clamp}(0.25 + 0.75 \cdot (1 - \text{playNowScore})). \end{aligned}$$

The threshold for `knight-card-playing-decision` is 0.90 because playing a Knight should be reserved for clearly valuable robber relief, resource need, or Largest Army progress.

A.3.5 BUILD-VS-SAVE-DECISION

This decision compares the expected value of spending now against preserving option value for later, incorporating immediate gains, future unlock potential, and seven-risk. The runtime constructs plan candidates for `road`, `settlement`, `city`, and `devCard`. Each plan uses the shared `baseScore` definitions from Section A.2.5, a cost vector, and a `missingCount`. `missingCount` is the total number of missing resource cards for the plan's cost vector. The discounted future value is

$$\text{effectiveScore} = \frac{\text{baseScore}}{1 + 0.75 \cdot \text{missingCount}}.$$

Each affordable build option gets $\text{score} = \text{clamp}(\text{baseScore})$. The save alternative is

$$B_{\text{eff}} = \text{best unaffordable effectiveScore}.$$

$$\text{saveScore} = \text{clamp}\left(\max(0.35, B_{\text{eff}} + S_7)\right),$$

where S_7 is 0.1 when $\text{resourceTotal} > 7$ and 0 otherwise.

The threshold for build-vs-save-decision is 0.90 because spend/save choices compare broad strategic options and should be close to the best current plan.

A.3.6 YEAR-OF-PLENTY-CARD-PLAYING-DECISION

Year of Plenty is strongest when it immediately unlocks a valuable build, repairs bottlenecks, and improves future resource flexibility. The runtime waits until both chosen resources are known, then scores every unordered resource pair using the shared plan table from Section A.2.5. It computes beforeBest as the largest baseScore affordable before the card, or 0 if no plan is affordable. It then simulates each candidate resource pair and computes afterBest the same way from the simulated post-card hand. For each pair:

$$\text{buildUnlock} = \text{clamp}(\max(0, \text{afterBest} - \text{beforeBest}) + U_{\text{imm}}),$$

where U_{imm} is afterBest if $\text{afterBest} > \text{beforeBest}$ and 0 otherwise.

$$\text{resourceScarcity} = \text{clamp}\left(\frac{\text{need}[r_1] + \text{need}[r_2]}{2}\right),$$

where r_1 and r_2 are the two resources in the candidate pair.

$$\text{longtermDiversityGain} = \text{clamp}\left(\frac{\text{distinctAfter} - \text{distinctBefore}}{2}\right).$$

The final score is

$$\begin{aligned} \text{Score} = & \text{clamp}(0.50 \cdot \text{buildUnlock} + 0.35 \cdot \text{resourceScarcity} \\ & + 0.15 \cdot \text{longtermDiversityGain}). \end{aligned}$$

The threshold for year-of-plenty-card-playing-decision is 0.90 because Year of Plenty should usually select resources that materially improve a near-term plan.

A.3.7 MONOPOLY-CARD-PLAYING-DECISION

Monopoly is strongest when opponents likely hold a concentrated resource, the acting player currently needs that resource, and the resulting swing unlocks a build. For each candidate called resource, the server sums opponents' actual holdings of that resource, computes beforeBest , simulates adding all stolen copies, and recomputes afterBest . It then uses:

$$\text{estimatedOpponentHoldings} = \text{clamp}\left(\frac{C_r}{C_{\text{max}}}\right),$$

where C_r is the total opponent copies of the called resource and C_{max} is the largest opponent holding count across any resource.

$$\text{personalResourceNeed} = \text{clamp}(\text{need}[r]).$$

$$\text{buildUnlock} = \text{clamp}(\max(0, \text{afterBest} - \text{beforeBest}) + U_{\text{imm}}),$$

where U_{imm} is afterBest if $\text{afterBest} > \text{beforeBest}$ and 0 otherwise. The resource score is

$$\begin{aligned} \text{resourceScore} = & \text{clamp}(0.55 \cdot \text{estimatedOpponentHoldings} + 0.30 \cdot \text{personalResourceNeed} \\ & + 0.15 \cdot \text{buildUnlock}). \end{aligned}$$

Monopoly plays emit a benchmark payload over the five possible resource calls. The implementation uses actual hidden-opponent holdings available on the server.

The threshold for monopoly-card-playing-decision is 0.90 because Monopoly calls should closely match the strongest resource concentration and personal need signal.

A.3.8 ROAD-BUILDING-CARD-PLAYING-DECISION

The central question is whether playing Road Building now is more valuable than holding it for a better timing window. The evaluator scores every currently legal road placement using the road-placement heuristic from Section A.3.3, sorts scores descending, averages the best two legal road scores (or one if only one exists), and computes

$$\text{playNowScore} = \text{clamp}(0.70 \cdot \text{topTwoAverage} + 0.30 \cdot \text{longestRoadLeverage}),$$

where

$$\text{longestRoadLeverage} = \text{clamp}\left(\frac{(L_{\text{self}} + 2) - L_{\text{opp}} + 3}{6}\right).$$

Here L_{self} is the player's current road length and L_{opp} is the best opponent road length. The hold alternative is

$$\text{holdScore} = \text{clamp}(0.30 + 0.70 \cdot (1 - \text{playNowScore})).$$

The threshold for road-building-card-playing-decision is 0.85 because Road Building timing has tactical flexibility and a slightly wider range of good play-now choices.

A.3.9 ROBBER-VICTIM-SELECTION

This task is evaluated as an option ratio over legal victims. Victim selection is a discrete choice whose value depends on leader pressure, theft value, and visible board strength. The visible leader is the player with the highest public score:

$$\text{visibleLeaderScore} = \text{visibleVP} + 2 \cdot \text{hasLongestRoad} + 2 \cdot \text{hasLargestArmy}.$$

If multiple players tie, the implementation keeps the first tied player in turn-order/index order. For each legal victim v , the score is

$$\text{victimScore}(v) = \text{clamp}(\text{leaderBase}(v) + \text{handValue}(v) + \text{visibleStrength}(v)),$$

where

$$\text{leaderBase}(v) = \begin{cases} 0.55, & \text{if } v \text{ is the visible leader,} \\ 0.25, & \text{otherwise,} \end{cases}$$

$$\text{handValue}(v) = \min\left(0.35, \frac{\text{resourceTotal}(v)}{20}\right),$$

$$\text{visibleStrength}(v) = \min\left(0.10, \frac{\text{visibleScore}(v)}{20}\right).$$

The threshold for robber-victim-selection is 0.90 because victim choice should usually target the best blend of leader pressure and theft value.

A.3.10 ROBBER-PLACEMENT

$$\begin{aligned} \text{Score} = & 0.35 \cdot \text{hurtLeader} + 0.30 \cdot \text{productionDamage} \\ & + 0.20 \cdot \text{theftValue} + 0.15 \cdot \text{selfHarmAvoidance}. \end{aligned}$$

For a candidate hex h , the evaluator uses the same visible leader definition from Section A.3.9. It identifies all opponents on that hex and computes each opponent's strength proxy:

$$\text{strength}(p) = \min\left(1, \frac{\text{visibleVP}(p) + \text{resourceTotal}(p)/8}{5}\right).$$

Let $w(h)$ be the dice-number probability weight of the hex. The damage terms are:

$$\text{hurtLeader} = \min\left(1, \frac{w(h) \sum_{p \in h, p=\text{visible leader}} \text{strength}(p)}{3}\right),$$

$$\text{productionDamage} = \min\left(1, \frac{w(h) \sum_{p \in h, p \neq \text{self}} \text{strength}(p)}{5}\right),$$

594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647

$$\text{theftValue} = \max_{p \in h, p \neq \text{self}} \min \left(1, \frac{\text{resourceTotal}(p)}{7} \right),$$

$$\text{selfHarmAvoidance} = \begin{cases} 0, & \text{if the acting player has a building on } h, \\ 1, & \text{otherwise.} \end{cases}$$

Hurting the leader is the anti-runaway term, production damage captures future income suppression, theft value captures immediate swing, and self-harm avoidance prevents obvious placements on the acting player's own production.

The threshold for robber-placement is 0.90 because robber placement is a direct tactical action and should strongly align with leader damage, production denial, and theft value.

A.3.11 ACCEPT-OR-REJECT-TRADE-OFFERS

$$\begin{aligned} \text{targetAcceptScore} = & 0.50 \cdot \text{selfGain} + 0.25 \cdot \text{leaderPenalty} \\ & + 0.15 \cdot \text{resourceUrgency} + 0.10 \cdot \text{fairnessBalance}. \end{aligned}$$

The current implementation approximates this as a two-option ratio over accept and reject. The accept score is the responder's simplified self-gain:

$$\text{acceptScore} = \text{selfGain} = \text{requestValue} - 0.5 \cdot \text{giveCost},$$

where requestValue uses the responder's need map and giveCost uses the responder's surplus map. The reject score is

$$\text{rejectScore} = \text{clamp}(1 - \text{acceptScore} + 0.1).$$

The threshold for accept-or-reject-trade-offers is 0.95 because accepting or rejecting a concrete offer should be nearly optimal to avoid rewarding harmful trades.

A.3.12 GENERATE-TRADE-OFFERS

$$\begin{aligned} \text{tradeOfferScore} = & 0.35 \cdot \text{legality} + 0.45 \cdot \text{selfGain} \\ & + 0.15 \cdot (1 - \text{leaderPenalty}) + 0.05 \cdot (1 - \text{fairnessPenalty}). \end{aligned}$$

A trade must first be legal. Its main purpose is to improve the acting player's position. Helping the leader is strategically risky, and highly unfair offers are less plausible and less likely to be accepted. The components are:

$$\begin{aligned} \text{selfGain} &= \text{clamp}(\text{requestValue} - 0.5 \cdot \text{giveCost}), \\ \text{fairnessPenalty} &= \min(1, \max(0, \text{giveCost} - \text{requestValue})), \\ \text{legality} &= \begin{cases} 0, & \text{if the offer gives resources the player does not own,} \\ 1, & \text{otherwise,} \end{cases} \\ \text{leaderPenalty} &= \begin{cases} 1, & \text{if the target is the visible leader,} \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

The generated offer is compared against the benchmark-best score via a ratio.

The threshold for generate-trade-offers is 0.70 because generated offers are exploratory and many legal, plausible offers may be useful even if not benchmark-best.

A.3.13 SELECT-TARGETED-TRADE-PARTNER

This task is evaluated as an option ratio. Each candidate partner is scored as

$$\text{partnerScore} = \text{selfGain} + \text{availabilityBonus} - 0.4 \cdot \text{leaderHelpPenalty},$$

where

$$\text{availabilityBonus} = \begin{cases} 0.25, & \text{if the partner can provide the requested resources,} \\ 0, & \text{otherwise,} \end{cases}$$

$$\text{leaderHelpPenalty} = \begin{cases} 1, & \text{if the partner is the visible leader,} \\ 0, & \text{otherwise.} \end{cases}$$

This favors partners that produce good value and can satisfy the request, while discounting trades with the leader. Targeted trade proposals emit this task at runtime, and the selected target is compared against all opponent candidates rather than only against itself. The threshold for `select-targeted-trade-partner` is 0.90 because targeted trade partner choice should closely match availability, self-gain, and leader-avoidance signals.

A.3.14 GENERATE-COUNTER-TRADE-OFFER

$$\text{counterTradeScore} = 0.35 \cdot \text{legality} + 0.45 \cdot \text{selfGain} + 0.15 \cdot (1 - \text{leaderPenalty}) + 0.05 \cdot (1 - \text{fairnessPenalty}).$$

The structural reasoning follows the `generated-trade-offer` task from Section A.3.12, but now from the responder's perspective relative to an incoming offer. The payload reuses the trade-quality sub-scores from Section A.3.12:

$$\begin{aligned} \text{selfGain} &= \text{clamp}(\text{requestValue} - 0.5 \cdot \text{giveCost}), \\ \text{fairnessPenalty} &= \min(1, \max(0, \text{giveCost} - \text{requestValue})), \\ \text{legality} &= \begin{cases} 0, & \text{if the responder gives resources they do not own,} \\ 1, & \text{otherwise,} \end{cases} \\ \text{leaderPenalty} &= \begin{cases} 1, & \text{if the original proposer is the visible leader,} \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

The benchmark payload stores this through `offerContext` with `bestOfferScore = 1` rather than logging each candidate counteroffer separately.

The threshold for `generate-counter-trade-offer` is 0.70 because counteroffers are exploratory like generated offers and should allow a broader set of reasonable proposals.

A.3.15 CITY-VS-SETTLEMENT-VS-ROAD-PRIORITIZATION

This task is evaluated as an option ratio. It compares build classes rather than board coordinates and asks which legal build family has the highest strategic value at the current state. Let the maxima below range over the currently legal options:

$$\begin{aligned} \text{settlementScore} &= \max \text{settlementScore}(v), \\ \text{roadScore} &= \max \text{roadScore}(e), \end{aligned}$$

where settlement scores use Section A.3.2 and road scores use Section A.3.3.

$$\text{cityScore} = 0.65 \cdot \text{production} + 0.10 \cdot \text{diversity} + 0.05 \cdot \text{portSynergy} + 0.20.$$

Production dominates because a city doubles an existing production site; diversity and port access still matter because they determine how valuable that amplification is, and the constant 0.20 reflects the guaranteed VP and output increase. The task is emitted for settlement, city, and normal road-building choices. Free roads from Road Building are intentionally excluded.

The threshold for `city-vs-settlement-vs-road-prioritization` is 0.90 because build-family prioritization should usually select the strongest currently legal development path.

A.3.16 DEVELOPMENT-CARD-PURCHASE-DECISION

This task is evaluated as an option ratio. Buying a development card is an investment choice against competing uses of resources and should be favored when hidden VP, Knight access, or tactical flexibility outweigh immediate board development. This decision is derived from the shared `devCard` plan `baseScore` from Section A.2.5:

$$\text{devCardBaseScore} = \max(0.2, 0.55 \cdot (0.5 + 0.5 \cdot \text{cityPlanScore}))$$

if the development deck is available, otherwise 0. The two options are

$$\begin{aligned} \text{buy-dev-card} &= \text{clamp}(\text{devCardBaseScore}), \\ \text{skip-dev-card} &= \text{clamp}(\max(\text{saveScore}, B_{\text{nondev}})), \end{aligned}$$

where B_{nondev} is the best affordable non-development-card plan `baseScore`. `buyDevCard` emits this task directly. If development-card purchase was affordable but skipped for the full turn, the negative case is recorded at turn end.

The threshold for `development-card-purchase-decision` is 0.95 because buying a development card is a precise resource-spending decision with clear competing alternatives.

A.3.17 DISCARD-STRATEGY-AFTER-SEVEN

This task is scored as

$$\frac{\text{RetainedHandValue}}{\text{BestRetainedValue}}.$$

Discarding is constrained by a fixed number of cards to lose, and the actual strategic objective is to maximize the value of the retained hand rather than the discarded hand. The evaluator enumerates all legal discard bundles summing to $\lfloor \text{total_hand_size}/2 \rfloor$, subtracts each from the current hand, and scores the retained hand with:

$$\begin{aligned} \text{bestAffordable} &= \max \text{ affordable plan baseScore,} \\ \text{bestFuture} &= \max \text{ plan effectiveScore,} \\ \text{diversity} &= \text{clamp} \left(\frac{\text{distinctPositiveResources}}{5} \right). \\ \text{retainedHandValue} &= \text{clamp}(0.55 \cdot \text{bestAffordable} + 0.30 \cdot \text{bestFuture} \\ &\quad + 0.15 \cdot \text{diversity}). \end{aligned}$$

The final task score is the selected retained-hand value divided by the best legal retained-hand value. Candidate discard bundles are generated directly from the player hand and required discard count. The threshold for discard-strategy-after-seven is 0.95 because discard choices are fully enumerated and should preserve nearly the best retained-hand value.

A.3.18 BANK-TRADE-DECISION

$$0.50 \cdot \text{resourceValueDelta} + 0.30 \cdot \text{buildUnlock} + 0.20 \cdot \text{opportunityCost}.$$

This task is currently approximated through a choice ratio over candidate bank/port trades plus a no-trade option. For each candidate trade, the evaluator computes the trade ratio using `GameLogic.getTradeRatio` and scores the candidate as

$$\text{tradeScore} = \text{clamp}(\text{need}[\text{get}] - 0.5 \cdot \text{surplus}[\text{give}] + 0.5),$$

and includes a fixed baseline

$$\text{no-trade} = 0.45.$$

Acquiring a needed resource is valuable, paying with a surplus resource is less painful than paying with a scarce one, and the no-trade baseline prevents weak legal trades from looking attractive by default. The named rubric components are not yet separately logged; they are compressed into a single trade desirability heuristic.

The threshold for bank-trade-decision is 0.85 because bank trades are deterministic but may have several near-equivalent resource-conversion options.

A.3.19 DECIDE-PURSUE-LONGEST-ROAD

$$\begin{aligned} \text{legacyScore} &= 0.40 \cdot \text{roadProgressRatio} + 0.30 \cdot \text{opponentRoadGap} \\ &\quad + 0.20 \cdot \text{resourceAffordability} + 0.10 \cdot \text{VPImpact}. \end{aligned}$$

The stronger formulation is to record a pending task episode, wait 4–6 turns for the evaluated player, and score using snapshot-plus-outcome evidence:

$$\begin{aligned} \text{pursueScore} &= 0.45 \cdot \text{blockingEffectiveness} + 0.30 \cdot \text{opponentProgressThreat} \\ &\quad + 0.15 \cdot \text{selfRoadSynergy} + 0.10 \cdot \text{resourceCost}. \end{aligned}$$

This redesign is preferable because Longest Road decisions are not fully observable from a single action. The live system already uses the delayed v2-style episode. At decision time it snapshots player road length, strongest opponent road rival and their road length, whether either player currently holds Longest Road, player visible score, and roads built so far in benchmark long-term stats.

After a four-turn horizon, decisive resolution, or game end, it computes:

$$\begin{aligned} G_{\text{self}} &= \max(0, L_{\text{self},\text{end}} - L_{\text{self},\text{start}}), \quad G_{\text{target}} = \max(0, L_{\text{target},\text{end}} - L_{\text{target},\text{start}}), \\ R_{\Delta} &= \max(0, \text{roadsBuiltSinceSnapshot}), \end{aligned}$$

```

756     opponentProgressThreat = clamp  $\left( \frac{(L_{\text{target,start}} - L_{\text{self,start}}) + 2H_{\text{target,start}} + 2}{8} \right)$ ,
757
758     blockingEffectiveness = clamp( $0.45 + 0.18G_{\text{self}} - 0.12G_{\text{target}} + 0.25H_{\text{self,end}}$ ),
759
760     selfRoadSynergy = clamp  $\left( \frac{G_{\text{self}}}{4} + 0.35H_{\text{self,end}} \right)$ .
761

```

Here each H indicator is 1 when that player has Longest Road at the named time and 0 otherwise. The resource-cost term is

```

764     resourceCost =  $\begin{cases} 1, & \text{successful defer and } R_{\Delta} = 0, \\ 0.5, & \text{pursue and } R_{\Delta} = 0, \\ \text{clamp} \left( \frac{G_{\text{self}} + \text{longestRoadBonus}}{R_{\Delta}} \right), & \text{otherwise.} \end{cases}$ 
765
766
767

```

The final alternatives are

```

769     pursueScore = clamp( $0.45 \cdot \text{blockingEffectiveness} + 0.30 \cdot \text{opponentProgressThreat}$ 
770          $+ 0.15 \cdot \text{selfRoadSynergy} + 0.10 \cdot \text{resourceCost}$ ),
771
772     deferScore = clamp( $0.45 \cdot (1 - \text{opponentProgressThreat})$ 
773          $+ 0.25 \cdot \left( 1 - \min \left( 1, \frac{G_{\text{target}}}{3} \right) \right)$ 
774          $+ 0.15 \cdot \left( 1 - \min \left( 1, \frac{G_{\text{self}}}{3} \right) \right) + 0.15D_0$ ).
775
776
777

```

Here D_0 is 1 if $R_{\Delta} = 0$ and 0.2 otherwise. This task now runs as a pending long-horizon episode, and finalization writes one ordinary benchmark task result after the horizon, decisive resolution, or game end.

The threshold for decide-pursue-longest-road is 0.95 because long-horizon race commitments are scored after an outcome window and should reflect strong evidence.

784 A.3.20 DECIDE-PURSUE-LARGEST-ARMY

```

785     pursueScore =  $0.40 \cdot \text{knightCardRatio} + 0.30 \cdot \text{opponentArmyGap}$ 
786          $+ 0.20 \cdot \text{devCardAffordability} + 0.10 \cdot \text{VPImpact}$ .
787

```

The rubric stated at the start of Section A.3.20 remains appropriate, but it should be evaluated over a 4–6 turn future window rather than immediately. Largest Army is not about one development-card purchase or one Knight; it is about whether a multi-turn commitment improves the player’s race standing without unacceptable economic damage. A hybrid intent-plus-outcome model is appropriate because pure outcome scoring would overreact to deck order and dice luck, while pure immediate scoring would ignore whether the strategic plan materialized. Like Longest Road, this task is implemented as a pending long-horizon episode. At decision time it snapshots player knights played, strongest opponent army rival and their knights played, whether either player currently has Largest Army, player visible score, player hand size, and development cards bought so far in benchmark long-term stats.

At finalization it computes:

```

798      $C_{\Delta} = \max(0, \text{devCardsBoughtSinceSnapshot})$ ,  $K_{\Delta} = \max(0, K_{\text{self,end}} - K_{\text{self,start}})$ ,
799
800      $G_{\text{start}} = K_{\text{self,start}} - K_{\text{target,start}}$ ,  $G_{\text{end}} = K_{\text{self,end}} - K_{\text{target,end}}$ ,
801
802     knightCardRatio =  $\begin{cases} 1, & K_{\Delta} > 0 \text{ and } C_{\Delta} = 0, \\ 0, & K_{\Delta} = 0, \\ \text{clamp} \left( \frac{K_{\Delta}}{C_{\Delta}} \right), & \text{otherwise,} \end{cases}$ 
803
804
805     opponentArmyGap = clamp  $\left( \frac{(G_{\text{end}} - G_{\text{start}}) + 3}{6} \right)$ ,
806
807      $\Delta R = \text{currentHandSize} - \text{snapshotHandSize}$ ,
808
809     devCardAffordability = clamp  $\left( 0.6 + \frac{\Delta R}{10} - 0.08C_{\Delta} \right)$ ,

```

810
$$\text{vpImpact} = \text{clamp} \left(\frac{\Delta \text{visibleVP} + H_{\text{army, end}}}{3} \right).$$

811

812 Here $H_{\text{army, end}}$ is 1 if the player has Largest Army at finalization and 0 otherwise. The final alterna-

813 tives are

814
$$\text{pursueScore} = \text{clamp}(0.40 \cdot \text{knightCardRatio} + 0.30 \cdot \text{opponentArmyGap}$$

815
$$+ 0.20 \cdot \text{devCardAffordability} + 0.10 \cdot \text{vpImpact}),$$

816

817
$$\text{deferScore} = \text{clamp}(0.35 \cdot (1 - \text{opponentArmyGap}) + 0.25 \cdot (1 - \text{knightCardRatio})$$

818
$$+ 0.20 \cdot R_{\text{safe}} + 0.20 \cdot (1 - H_{\text{army, end}})).$$

819

820 Here R_{safe} is 1 if $\Delta R \geq 0$ and 0.4 otherwise.

821 The threshold for `decide-pursue-largest-army` is 0.95 because Largest Army pursuit is a

822 long-horizon commitment and should be supported by strong snapshot-plus-outcome evidence.

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863